

《数据结构》复习要点

一、绪论

1. 数据结构的研究内容

包括数据的逻辑结构、数据的存储结构和对数据元素施加的操作（即数据的运算）三方面。

2. 算法概念及其评价

五大特性：有穷性、确定性、可行性、输入性、输出性；

算法评价：正确性、可读性、健壮性、高效性、低存储性；

要特别弄清诸如语句频度、时间复杂度、空间复杂度等的概念，评价一个算法好坏的两个主要标准——时间复杂度和空间复杂度。

3. 算法和程序的区别

a. 程序不一定满足有穷性(如操作系统)；

b. 程序中的指令必须是机器可执行的，算法中的指令则无此限制；

c. 算法代表了对问题的解，程序则是算法在计算机上的特定的实现（一个算法若用程序设计语言来描述，它才是一个程序）；

d. 数据结构+算法=程序。

4. 数据结构的相关概念

数据结构包括数据的逻辑结构和数据的存储结构。

数据元素是组成数据的基本单位。

数据项是数据不可分割的最小单位。

数据逻辑结构的两大类型（线性结构、非线性结构）、抽象数据类型（是指一个数学模型以及定义在该模型上的一组操作）。

数据的四种逻辑结构（集合、线性、树形、图形）

数据的四种存储结构（顺序、链式、索引、散列）

不管是顺序存储结构还是链式存储结构，都要存储数据元素本身和数据元素之间的关系
顺序存储结构、链式存储结构都可存各种逻辑结构

5. 数据逻辑结构、存储结构的区别和联系

区别：数据的逻辑结构是一个数学模型；数据的存储结构是数据的逻辑结构在计算机内部的存储方式。

联系：一种逻辑结构可以用多种存储结构来存储；一种存储结构可以用来存多种逻辑结构。

二、线性表

1. 线性表的定义

从数据对象、元素间的关系、基本操作三方面进行阐述。

2. 线性表的两种存储结构及其优缺点

线性表的顺序存储是指在内存中用地址连续的一块存储空间对线性表中的各元素按其逻辑顺序依次存放，用这种存储形式存储的线性表称为顺序表。线性表的链式存储结构是用一组任意的存储单元存储线性表的各个数据元素。为了表示线性表中元素的先后关系，每个元素除了需要存储自身的信息外还需保存直接前趋元素或直接后继元素的存储位置。所以，链式存储结构的线性表其元素之间的逻辑关系是通过结点的指针域来表示的。

顺序存储有三个优点：

- (1) 方法简单，用数组，容易实现。
- (2) 不用为表示结点的逻辑关系而增加额外开销。
- (3) 具有按元素序号随机访问的特点。

两个缺点：

- (1) 进行插入、删除时，平均移动表中一半的元素，效率较低。
- (2) 需预先分配足够大的存储空间。空间估计过大，会导致空间闲置（造成浪费）；预先分配过小，又会造成溢出。

链表的优缺点恰好与顺序表相反。

与线性表两种存储结构有关的知识有：线性表中逻辑上相邻的两个元素其存放位置不一定相邻；线性表的链式存储表示不一定优于顺序存储表示；链式存储方式以指针表示结点间的逻辑关系；一个需经常作插入、删除运算的线性表应采用链式存储结构；线性表元素个数稳定、很少进行插入和删除操作应采用顺序存储结构；若线性表最常用的操作是存取第 i 个元素及其前驱元素的值，则采用顺序存储方式最节省运算时间；在顺序存储结构、链式存储结构上删除元素的平均时间复杂度。

3. 线性表的建立、插入、删除、输出

顺序表的类型定义、插入、删除，单链表的建立（头插建立、尾插建立）、插入（后插的基本操作）、删除、输出历来都是重点。

如删除顺序表中某元素多少元素需移动、在顺序表中插入一个元素有多少元素需移动等。

再如在一个单链表的 p 所指的结点之后插入一个 s 所指的结点的操作、根据单链表结点类型定义写出带头结点单链表的升序插入、在带头结点单链表中查找元素并返回其地址、删除带头结点单链表中某元素、统计带头结点单链表中元素个数算法（函数）。

例 1：设有一带头结点的数据域为整型数据的单链表 h，给出单链表结点类型定义，并设计算法输出单链表中的所有元素。

```
typedef struct node
{
    int data;
    struct node *next;
} slnode;

void output(slnode *h)
{
    slnode *p;
    p=h->next;
    while(p!=null)
    {
        printf("%d",p->data);
        p=p->next;
    }
}
```

例 2 设有一带头结点的数据域为整型数据的单链表 H，给出单链表结点类型定义，并设计算法统计单链表中元素 x 的个数。

```
typedef struct node
{
    int data;
    struct node *next;
} slnode;

int tj(slnode *H)
{
    slnode *p;
    int n=0;
    p=H->next;
    while(p!=NULL)
    {
        if (p->data==x)  n++;
        p=p->next;
    }
    return n;
}
```

4. 单链表头结点的作用
简化运算。

三、栈和队列

1. 栈和队列的概念、特点

如：栈和队列是特殊的线性表

栈和队列的存储方式，既可以是顺序方式，又可以是链式方式。

同一栈内各元素的类型必须一致

栈是限定仅能在表尾一端进行插入、删除操作的线性表；

队列是限定仅能在表头进行删除、表尾进行插入的线性表；

栈的特点是后进先出；

队列的特点是先进先出。

若将 1, 2, 3 按先后次序入栈，出栈顺序不可能为 3, 1, 2

在作进栈运算时应先判别栈是否已满，在作退栈运算时应先判别栈是否已空，当栈中元素为 n 个，作进栈运算时发生上溢，则说明该栈的最大容量为 n 。

2. 顺序栈、链栈、循环队列、链队列

如循环队列解决什么问题？假溢出

循环队列队空、队满判断：

在少用一个存储单元的前提下

$front == rear$ 时队空；

$(rear + 1) \% maxsize == front$ 时队满。

例：循环队列用数组 $A[0..m-1]$ 存放其元素值，已知其头尾指针分别为 $front$ 和 $rear$ ，则当前元素个数为 $(rear - front + m) \% m$ ；

为了解决普通顺序存储结构队列的“假溢出”现象，节约内存单元，通过在队列操作中加入数学中的求余运算，可以将其构造成循环队列；

在循环队列中，为了能够区分队满和队空，往往少用一个元素空间。在这种情况下，队满的条件是尾指针+1等于头指针。

3. 什么情况下要用栈？什么情况下要用队列？

后进先出；先进先出。

如括号匹配、图的深度遍历等用栈实现。

二叉树的层次遍历、图的广度遍历等用队列实现。

例：解决计算机与打印机之间速度不匹配问题，须设置一个数据缓冲区，应是一个队列结构。

4. 栈容量大小判断

如设栈 S 和队列 Q 的初始状态均为空，元素 e1,e2,e3,e4,e5,e6 依次通过栈 S,一个元素出栈后即进入队列 Q，若 6 个元素出队序列为 e2,e4,e3,e6,e5,e1，则栈 S 的容量至少为多少？（至少为 3）。

5. 对同一问题的求解程序，递归程序无论时间还是空间都不如非递归程序。

例：对同一问题的求解程序，递归程序比非递归程序要花费更多的时间；
任何一个递归过程都可以转换成非递归过程；

四、树结构

1. 树和二叉树的区别

当只有一个子树时，二叉树有序，树无序。

例 度不大于二的树不是二叉树。

2. 几种特殊形态的二叉树

满二叉树、完全二叉树、排序二叉树的概念及区别。

例：完全二叉树不是最优的二叉树

3. 二叉树性质

重点掌握： 2^{i-1} , 2^k-1 , $n_0=n_2+1$, $\lfloor \log_2 n \rfloor +1$

即若二叉树的层次从 1 开始，则在二叉树的第 $i (i \geq 1)$ 层最多有 2^{i-1} 个结点；

深度为 K ($K \geq 1$) 的二叉树至多有 2^k-1 个结点；

具有 n 个结点的完全二叉树的深度为 $\lfloor \log_2 n \rfloor +1$ ；

4. 二叉树存储结构

顺序存储结构、二叉链存储结构、三叉链存储结构。

5. 二叉树建立及其四种遍历方法

先、中、后序递归遍历及层次遍历，不仅要掌握遍历过程、会根据二叉树写出某种遍历结果，而且要会根据两种遍历结果（先序、中序或者中序、后序）画出二叉树。注意，给出先序和后序不能唯一地确定一棵二叉树。也即任意给出二叉树两种遍历结果不一定能把二叉树恢复出来。

例：已知一棵二叉树的先序遍历为 cedba，中序遍历为 debac，请画出该二叉树，并写出后序遍历结果

6. 线索二叉树（中序、先序、后序）概念

7. 二叉树遍历的应用

如写出二叉树结点个数、叶子结点个数、度为 2 的结点个数统计，二叉树深度计算，查找二叉树中结点（返回地址）算法（函数）等。

例：设二叉树的数据类型为结构体类型 datatype，给出二叉链表存储二叉树的结点类型定义，并写出统计二叉树 t 中结点总数的算法。

```
typedef struct node
{ datatype data;
  struct node *lchild, *rchild;
} bitree;
```

第一种方法（使用全局变量）：

```
int n=0;
void tj(bitree *t)
{ if(t!=NULL)
  { n++;
    tj (t->lchild);
    tj (t->rchild);
  }
}
```

第二种方法（不用全局变量）：

```
int tj (bitree *t)
{ if(t==NULL) return 0;
  else return (1+ tj (t->lchild)+ tj (t->rchild));
}
```

8. 哈夫曼树(最优二叉树)概念、总结点数（具有 n 个叶子结点的哈夫曼树有 $2n-1$ 个结点）、构造、哈夫曼编码、WPL 值计算

例：设给定一个权值集合 $W=(1, 3, 5, 8, 10, 11)$ ，要求根据给定的权值集合构造一棵哈夫曼树并计算哈夫曼树的带权路径长度 WPL

假设用于通讯的电文由 6 个字母 {A, B, C, D, E, F} 组成，各字母在电文中出现的频率分别为 7, 19, 5, 6, 32, 3，试为这 6 个字母设计哈夫曼树并写出对应的哈夫曼编码。约定频率小的在左边，大的在右边。

9. 树的存储结构、遍历（无中序）及树转化为二叉树

树的存储结构：双亲表示法、双亲孩子表示法、孩子-兄弟表示法；树转化为二叉树后，相应的二叉树根结点无右子树；当以二叉链表作树的存储结构时，树的先根遍历可借用二叉树的先根遍历的算法实现，树的后根遍历可借用二叉树的中序遍历的算法实现等。

例：树是图的一种特殊情况；

对树而言，不合适的遍历是中序；

把任一棵树转化为二叉树，该二叉树的右子树必空；

森林 F 中有三棵树，节点数目分别为 m_1 、 m_2 、 m_3 ，与 F 对应的二叉树结构的根节点的右子树的节点数为 m_2+m_3 ；

用孩子兄弟法存储树时，树的先序遍历结果与其对应的二叉树的先序遍历结果相同，树的后序遍历结果与其对应的二叉树的中序遍历结果相同。

五、图结构

1. 图的概念

图和树的区别，四种图——有向图、无向图、有向网、无向网的区别；在一个具有 n 个顶点的无向完全图中，所含的边数为 $n*(n-1)/2$ ，有 n 个顶点的强连通图最多有 $n(n-1)$ 条边（有向边、弧），最少有 n 条边等。

2. 图的邻接矩阵、邻接链表存储结构

如 n 个顶点 e 条边的无向图的邻接表的存储中，表头结点的个数是 n ，边结点的个数是 $2e$ ；无向图的邻接矩阵具有对称性；带权有向图 G 用邻接矩阵 A 存储，则顶点 i 的入度等于 A 中第 i 列非 ∞ 且非 0 的元素个数，顶点 i 的出度等于 A 中第 i 行非 ∞ 且非 0 的元素个数；从有向图中删除一条边 $\langle i, j \rangle$ ，其邻接矩阵第 i 行第 j 列要改为 0；从无向图中删除一条边 (i, j) ，其邻接矩阵的第 i 行第 j 列和第 j 行第 i 列都要改为 0；已知一个有向图的邻接矩阵，要想删除所有以第 i 个顶点为起始点的弧，应该将邻接矩阵的第 i 行置零；设有向图 G 中有 n 个顶点和 e 条边，则其对应的邻接表中有 e 个边结点等。

3. 图的遍历

深度和广度遍历，分别用栈和队列实现。

例：图的广度优先搜索利用队列实现；

图的深度优先搜索遍历算法要用栈来实现；

画出图邻接矩阵存储结构，并根据邻接矩阵写出从顶点 A 开始分别进行深度优先搜索遍历和广度优先搜索遍历的结果；

画出图的邻接链表存储结构，并写出根据该邻接链表从顶点 A 开始进行深度优先搜索和广度优先搜索遍历的结果。

4. 最小生成树的概念及 Prim 和 Kruskal 两种方法生成最小生成树过程

最小生树不是边最少的生成树，而是所有 $n-1$ 条边的权值之和最小的生成树。若这个连通图有 n 个顶点，则它的生成树有 $n-1$ 条边；如果含 n 个顶点的图形形成一个环，则它有 n 棵生成树；普里姆（Prim）算法适合于边稠密网；克鲁斯卡尔（Kruskal）算法适合于边稀少的网。图的最小生成树的形状可能不唯一。

不仅要会根据网图由 Prim 或 Kruskal 方法生成最小生成树，而且还要会根据 (V, E) （顶点集合 $V=\{1, 2, 3, \dots\}$ 和边集合 $E=\{(1, 2) 8, (2, 3) 5, \dots\}$ 由 Prim 或 Kruskal 方法从某顶点开始构造生成最小生成树。Prim 算法的时间复杂度为 $O(n^2)$ 。

5. 求最短路径的 Dijkstra 和 Floyd 方法

迪杰斯特拉（Dijkstra）算法求最短路径的基本思想是按路径长度（这里的路径长度指路径上边或者弧的权值之和）递增的次序产生最短路径，其时间复杂度为 $O(n^2)$ 。Floyd 方法时间复杂度为 $O(n^3)$ 。要弄清求最短路径的过

程。

六、查找

1. 三种查找方法及其区别、比较

	顺序查找	折半查找	分块查找
存储结构	顺序、链式	顺序	索引存储
有序性	有序、无序	有序	按块有序
ASL	最大	最小	居中
时间复杂度	$O(n)$	$O(\log_2 n)$	$O(m+t)$

如顺序查找平均比较次数等

2. 使用折半查找前提、折半查找算法及其判定树

有序、顺序存储。

例：折半查找只能在有序的顺序表上进行；

画出描述 $n=8$ （具有 8 个元素）的折半查找（即二分查找）过程判定树，并计算查找成功时的平均查找长度 ASL（假定查找每个元素的概率相等）；

对于一个具有 100 个元素的有序表，要用二分查找（即折半查找）查找某个元素，最多需要比较 7 次。

折半查找其判定树应用，如有 1000 个元素最多比较多少次（10 次）。

3. 排序二叉树（又称二叉排序树）的概念及其构造（插入）、查找、输出、删除某节点（还要保证其性质）

例：如果在一棵二叉树中，每个结点的值都大于它的左子树上所有结点的值，而小于它的右子树上所有结点的值，这样的一棵二叉树称为排序二叉树（或二叉排序树）

依次输入关键字 {50, 28, 73, 91, 56, 18, 34, 86}，画出所生成的二叉排序树，并写出其中序遍历序列。

4. 哈希表构造及哈希查找过程

掌握常用哈希函数——直接定址法和除留余数法，常用处理冲突方法——线性探测法、二次探测法和拉链法及 ASL 计算。

例：在哈希函数 $H(k) = k \% m$ （“%”表示求余）中，一般来讲， m 应取质数（素数）；设一组初始记录关键字集合为 (25, 10, 8, 27, 32, 68)，哈希表地址区间为 0~7，哈希函数 $H(k) = k \bmod 7$ ，用线性探测再散列作为解决冲突的方法设计哈希表，将其内容填入下表（表 1），并计算查找成功时的平均查找长度 ASL；

表 1（哈希表）

地址	0	1	2	3	4	5	6	
关键字								

设哈希表长 $m=14$ ，哈希函数 $H(\text{key}) = \text{key} \bmod 11$ 。表中已有四个记录：

addr(15)=4, addr(38)=5, addr(61)=6, addr(84)=7, 其余地址为空。若采用二次探测再散列处理冲突, 关键字为 49 的记录地址是 9。

5. 静态查找与动态查找的区别

看有无插、删操作。

例:

静态查找表与动态查找表的根本区别在于施加在其上的操作不一样

七、排序

重点掌握各种排序方法(直插、折半插入、希尔、冒泡、快速、简选、堆排、二路归并, 基数排序了解)的排序过程及其特点(时间复杂度、稳定性、空间复杂度), 尤其要弄清直插、冒泡、快速和堆排序。

如要知道排序按记录的次关键码进行, 一般应选择稳定的排序方法; 堆排序属于选择排序; 在待排序的元素序列基本有序的前提下, 效率最高的排序方法是直接插入排序和冒泡排序; 直接插入排序、折半插入排序的效率与初始数据有关; 快速、堆、冒泡、希尔排序算法中, 空间复杂度最大的是快速排序; 既稳定又省时间的排序算法是归并排序; 冒泡排序在某一趟排序过程中若不存在交换就不需要下一趟排序了(就应退出循环)。会写直接插入排序、冒泡排序(升序或者降序, 不特别指明为升序)等基本算法。给出一组数据, 会写出初始堆(大顶堆或小顶堆, 不特别指明为小顶堆)等。

例: 稳定的排序方法是指在排序中, 关键字值相等的不同记录间的前后相对位置保持不变; 直接选择排序是一种不稳定的排序方法。

设有 1000 个无序的数据元素, 希望用最快的速度找到前 10 个最大元素, 最好选用堆排序; 在已知待排序文件已基本有序的前提下, 效率最高的排序方法是直接插入排序;

若排序按记录的次关键码进行, 一般应选择稳定的排序方法;

希尔排序法、快速排序法、堆排序法和二路归并排序法四种排序法中, 要求辅助空间最多的是二路归并;

若需要在 $O(n\log n)$ 的时间内完成对一组数的排序, 且要求排序是稳定的, 则可选的排序方法是归并排序。

对 n 个元素采取冒泡排序, 最少的比较次数是 $n-1$;

用堆排序实现升序排列时, 建立的初始堆必须是大顶堆;

对于一组给定的关键字{49, 38, 27, 15, 94, 53}, 写出以第一个关键字作为基准元素运用快速排序方法进行升序排序时第一次划分的结果, 如果来实现排序, 共需进行几次划分;

(要全面复习，掌握重点)

题 型

- 一、 单项选择题
- 二、 判断题
- 三、 填空题
- 四、 简答题
- 五、 算法设计题